

zVault

Smart Contract Audit

Contents

Revision History & Version Control

1.0 Disclaimer

2.0 Overview

- 2.1 Project Overview
- 2.2 Scope
- 2.3 Project Summary
- 2.4 Audit Summary
- 2.5 Security Level References
- 2.6 Vulnerability Summary

3.0 Executive Summary

- 3.1 Findings
- 3.2 Recommendations

4.0 Technical Analysis

- 4.1 Redemption Request Rejection locks User Funds
- 4.2 Role Hash Mismatch Between Access Control and Token Contracts
- 4.3 Fiat Redemption Approval Reverts Due to Allowance Check on Zero Address
- 4.4 Non Standard Decimal Scaling Leads to Zero Value Deposits
- 4.5 PriceOracle Interface Mismatch (feedAdminRole() Missing)
- 4.6 Sanctioned Users Can Bypass Bans via Token Transfers
- 4.7 Vaults Use Stale Prices if Oracle Stops Updating
- 4.8 Redemption Request Can Record Zero Rates, Breaking Safe Approval
- 4.9 Redundant and Confusing mHYPER.sol Contract File
- 4.10 Setting Price Tolerance to Zero Disables Safety Check Instead of Enforcing Exact Price
- 4.11 Admin approveRequest Bypasses Price Oracle Safety

5.0 Auditing Approach and Methodologies applied

6.0 Limitations on Disclosure and Use of this Report

Revision History & Version Control

Start Date	End Date	Author	Comments/Details
16 January 2026	02 February 2026	Shashank Wangkar	Interim Release for the Client

Reviewed by	Released by
Nishita Palaksha	Nishita Palaksha

Entersoft was commissioned to perform a source code review on zVault smart contracts. The review was conducted between January 16, 2026, to February 02, 2026. The report is organized into the following sections.

- Executive Summary: A high level overview of the security audit findings.
- Technical analysis: Our detailed analysis of the Smart Contract code

The information in this report should be used to understand overall code quality, security, correctness, and meaning that code will work as described in the smart contract.

1.0 Disclaimer

This is a limited audit report on our findings based on our analysis, in accordance with good industry practice as at the date of this report, in relation to: (i) smart contract best coding practices and vulnerabilities in the framework and algorithms based on white paper, code, the details of which are set out in this report, (Smart Contract audit). To get a full view of our analysis, it is crucial for you to read the full report. While we have done our best in conducting our analysis and producing this report, it is important to note that you should not rely on this report and cannot claim against us based on what it says or does not say, or how we produced it, and it is important for you to conduct your own independent investigations before making any decisions. We go into more detail on this in the disclaimer below – please make sure to read it in full.

DISCLAIMER: By reading this report or any part of it, you agree to the terms of this disclaimer. If you do not agree to the terms, then please immediately cease reading this report, and delete and destroy any copies of this report downloaded and/or printed by you. This report is provided for information purposes only and on a non-reliance basis and does not constitute investment advice. No one shall have any right to rely on the report or its contents, and Entersoft Australia and its affiliates (including holding companies, shareholders, subsidiaries, employees, directors, officers, and other representatives) (Entersoft) owe no duty of care towards you or any other person, nor does Entersoft make any warranty or representation to any person on the accuracy or completeness of the report. The report is provided "as is", without any conditions, warranties or other terms of any kind except as set out in this disclaimer, and Entersoft hereby excludes all representations, warranties, conditions and other terms (including, without limitation, the warranties implied by law of satisfactory quality, fitness for purpose and the use of reasonable care and skill) which, but for this clause, might have effect in relation to the report. Except and only to the extent that it is prohibited by law, Entersoft hereby excludes all liability and responsibility, and neither you nor any other person shall have any claim against Entersoft, for any amount or kind of loss or damage that may result to you or any other person (including without limitation, any direct, indirect, special, punitive, consequential or pure economic loss or damages, or any loss of income, profits, goodwill, data, contracts, use of money, or business interruption, and whether in delict, tort (including without limitation negligence), contract, breach of statutory duty, misrepresentation (whether innocent or negligent) or otherwise under any claim of any nature whatsoever in any jurisdiction) in any way arising from or connected with this report and the use, inability to use or the results of use of this report, and any reliance on this report. The analysis of the Smart contract is purely based on the smart contract code shared with us alone.

2.0 Overview

2.1 Project Overview

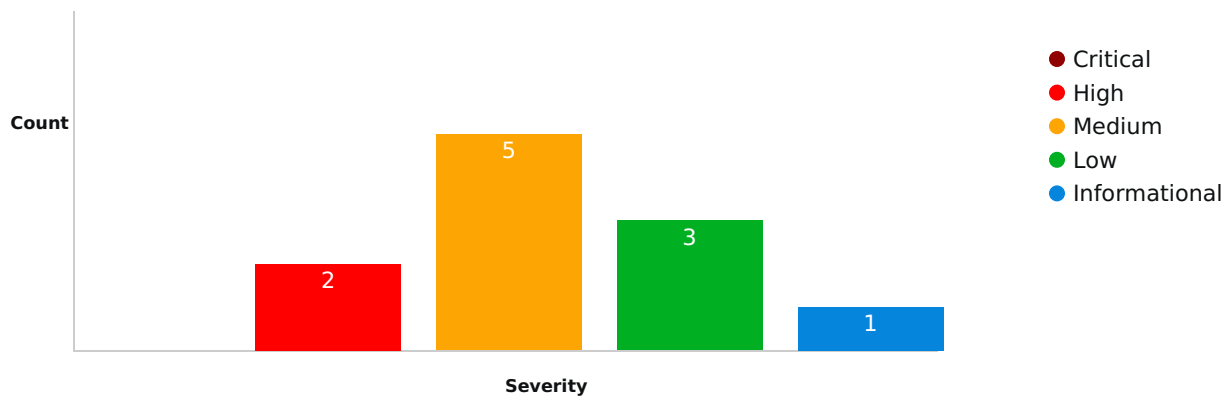
During the period of **16 January 2026 to 02 February 2026**, Entersoft performed smart contract security audits for **zVault**.

2.2 Scope

The scope of this audit was to analyze and document the smart contract codebase for quality, security, and correctness.

File In Scope:

- ManageableVault.sol
- ManageableVaultRedeem.sol
- WithSanctionsList.sol
- ZothInitializable.sol
- Blacklistable.sol
- FunctionsAccessControl.sol
- Greenlistable.sol
- Pausable.sol
- WithFunctionsAccessControl.sol
- WithZothAccessControl.sol
- ZothAccessControl.sol
- ZothAccessControlRoles.sol
- IDataFeed.sol
- IDepositVault copy.sol
- IDepositVault.sol
- IManageableVault.sol
- IManageableVaultRedeem.sol
- IRedemptionVault.sol
- ISanctionsList.sol
- IVaultShared.sol
- IZToken.sol
- DecimalsCorrectionLibrary.sol
- mHYPER.sol
- ERC1967Proxy.sol
- DepositVault.sol
- PriceOracle.sol
- RedemptionVault.sol
- ZHyperDepositVault.sol
- ZHyperZothAccessControlRoles.sol
- zHYPER.sol



3.0 Executive Summary

Entersoft completed a comprehensive technical audit of the zVault smart contracts from January 16 to February 02, 2026. The primary objective was to identify vulnerabilities and security risks, ensuring the contracts meet industry standards for security, reliability, and performance. Our approach combined cutting-edge static analysis with in-depth manual security analysis, deliberately avoiding sole reliance on automated tools. This thorough assessment identified a number of vulnerabilities, detailed in the summary table.

Testing Methodology

We have leveraged static analysis techniques extensively to identify potential vulnerabilities automatically with the aid of cutting edge tools such as Slither and Aderyn. Apart from this, we carried out extensive manual testing to iron out vulnerabilities that could slip through an automated check. This included a variety of attack vectors like reentrancy attacks, overflow and underflow attacks, timestamp dependency attacks, and more. While going through the due course of this audit, we also ensured to cover edge cases, and built a combination of scenarios to assess the contracts resilience.

Our testing methodology in Solidity adhered to industry standards and best practices, integrating partially implemented OWASP and NIST SP 800 standards for encryption and signatures. Solidity's renowned security practices were complemented by tools such as Solhint for linting, and the Solidity compiler for code optimization. Sol-profiler, Sol-coverage, and Sol-sec were employed to ensure code readability and eliminate unnecessary dependencies.

Tools Used for Audit:

In the course of our audit, we leveraged a suite of tools to bolster the security and performance of our program. While our team drew on their expertise and industry best practices, we also integrated various tools into our development environment. Noteworthy among them are Slither and Aderyn. This holistic approach ensures a thorough analysis, uncovering potential issues that automated tools alone might overlook. Entersoft takes pride in utilizing these tools, which significantly contribute to the quality, security, and maintainability of our codebase.

Code Review / Manual Analysis:

Our team conducted a manual analysis of the Solidity contracts to identify new vulnerabilities or to verify vulnerabilities found during static and manual analysis. We carefully analyzed every line of code and made sure that all instructions provided during the onboarding phase were followed. Through our manual analysis, we were able to identify potential vulnerabilities that may have been missed by automated tools and ensure that the smart contract was secure and reliable.

Documentation Overview

Documentation is crucial for any project as it ensures clarity and understanding, helping developers and users grasp how the code and protocol function. It aids in new contributors, auditors facilitating maintenance and troubleshooting by providing insights into design decisions.

For blockchain protocols, thorough documentation is particularly important due to the complexity and need for security and transparency.

Below is the required documentation, rated according to the quality as provided by the protocol.

Protocol Documentation:

- **Purpose:** To describe the overall protocol, its goals, architecture, and how it operates.
- **Quality Provided:** Good

Technical Documentation:

- **Purpose:** To provide detailed information about the technical aspects of the protocol, including smart contracts, APIs, and their interactions.
- **Quality Provided:** Good

Inline Comments:

- **Purpose:** To explain specific parts of the code, such as the purpose of functions, variables, and logic.

Quality Provided: Good

Smart Contract Audit Testing objectives	Result
OVERALL SECURITY POSTURE	● FAIR

3.1 Findings

Vulnerability ID	Contract Name	Severity	Status
1	RedemptionVault	● High	Acknowledged
2	ZothAccessControl	● High	Fixed
3	RedemptionVault	● Medium	Acknowledged
4	DepositVault	● Medium	Fixed
5	IDataFeed	● Medium	Fixed
6	zHYPER	● Medium	Fixed
7	PriceOracle	● Medium	Fixed
8	RedemptionVault	● Low	Acknowledged
9	mHYPER	● Low	Fixed
10	PriceOracle	● Low	Acknowledged
11	DepositVault	● Informational	Acknowledged

3.2 Recommendations

Entersoft's review indicates the zVault protocol has a clear architectural design and aligns with its intended vault based business model. The system cleanly separates access control, vault logic, token contracts, and the oracle layer. The codebase is generally readable and modular, with consistent use of shared abstractions (ManageableVault/ManageableVaultRedeem), and the repository layout is organized and easy to navigate. However, a number of logic and operational gaps must be addressed before production deployment.

Key findings include:

Critical and High Severity Issues: The most urgent risks stem from a severe configuration mismatch in Access Control that renders minting and pausing inoperable upon deployment (H-1), and a logic flaw in the Redemption Vault that locks user funds when a request is rejected (H-2). Additionally, the default approval mechanism bypasses Oracle safety checks (H-3), creating a single point of failure if admin keys are compromised.

Medium Severity Findings: The audit highlighted integration fragilities, including a Decimal Scaling Trap that causes loss of value for native inputs , a bypass of sanctions enforcement during token transfers , and multiple Oracle integration issues (staleness checks, interface compliance, and zero tolerance configuration traps). The Fiat Redemption feature is also functionally broken by default due to incorrect allowance logic.

Code Quality & Architecture:

Architecture: The codebase follows a clean, modular design pattern separating Assets, Gateways (Vaults), and Data (Oracle). The abstract base contracts used correctly for reuse of code.

Code Quality: While individual contract logic is generally readable, the system exhibits signs of fork and modify development that lacked a final cleanup pass. This is evidenced by the presence of redundant dead code (mHYPER file), the "M_TBILL" role error, and missing interface implementations in the Oracle.

Logic Quality: Core math is handled safely via libraries, but business logic regarding edge cases (e.g., zero rates, oracle outages, fiat settlement) is fragile and prone to Denial of Service (DoS).

The zVault Protocol exhibits a strong conceptual foundation as a DeFi product. The issues identified during the assessment were predominantly logical and configuration-related, rather than indicative of fundamental flaws in the core architecture. Remediation of the Critical and High severity findings, particularly those that could lead to fund lockups or liveness failures, along with improvements to the Medium severity integration gaps, positions the protocol with a security posture appropriate for mainnet deployment.

All reported findings have either been fully remediated or formally acknowledged by the client, with the associated risks explicitly accepted. Based on the final assessment, the protocol has been assigned an overall security rating of **FAIR**.

4.0 Technical Analysis

4.1 Redemption Request Rejection locks User Funds

Severity	Status	Type of Analysis
● High	Acknowledged	Dynamic

Contract Name:

RedemptionVault

Description:

The `RedemptionVault` implements a "Request Approval" pattern for users willing to redeem `zHYPER` for stablecoins.

1. When a user calls `redeemRequest`, the protocol immediately transfers the `zHYPER` tokens from the user to the `RedemptionVault` contract.
2. The request enters a `Pending` state.
3. If the Admin decides to reject this request (e.g., due to KYC flags or limits) by calling `rejectRequest`, the function updates the request status to `Canceled`.

The `rejectRequest` function **does not refund the tokens**. It simply marks the request as dead. The `zHYPER` tokens remain inside the Vault contract balance, and the user has no way to call `withdraw` or `claim` to get them back.

Locations:

- [RedemptionVault.sol:252-259](#)

Remediation:

Update the `rejectRequest` function to push the trapped tokens back to the user immediately when the request is canceled.

example code fix :

```
function rejectRequest(uint256 requestId) external onlyVaultAdmin {
    Request memory request = redeemRequests[requestId];
    _validateRequest(request.sender, request.status);
    // 1. Update status
    redeemRequests[requestId].status = RequestStatus.Canceled;
    // 2. Return the tokens to the user
    // zToken uses 18 decimals and adheres to IERC20
    IERC20(address(zToken)).safeTransfer(
        request.sender,
        request.amountZToken
    );
    emit RejectRequest(requestId, request.sender);
}
```

Client's Comments:

Risk accepted by the Client . They opted not to change the contract, preferring to retain the ability to lock malicious funds and handle benign refunds via separate, manual transactions.

Zoth can refund in another transaction if required. This maintains flexibility for handling edge cases with malicious actors.

Impact:

- The user loses custody of their assets immediately upon requesting. Upon rejection, they do not regain custody.
- The protocol administrators become solely responsible for manually calculating and returning these funds. This creates a high touch operational burden.
- If the Admin loses access to the keys capable of calling `withdrawToken` (the emergency function), the user's funds are lost forever.

Code Snippet:

```
function rejectRequest(uint256 requestId) external onlyVaultAdmin {

    Request memory request = redeemRequests[requestId];

    _validateRequest(request.sender, request.status);

    redeemRequests[requestId].status = RequestStatus.Canceled

    emit RejectRequest(requestId, request.sender);

}
```

Reference:

NA

POC's Description:

NA

Proof of Vulnerability:

N.A.

4.2 Role Hash Mismatch Between Access Control and Token Contracts

Severity	Status	Type of Analysis
● High	Fixed	Dynamic

Contract Name:

ZothAccessControl

Description:

The vaults access control initialization logic contains a severe configuration error due to a likely fork of code from a previous codebase (referencing "M_TBILL").

In `ZothAccessControl.sol`, the internal `_setupRoles` function is responsible for granting the initial operational permissions to the admin during deployment. Currently, this function grants the following roles:

- `M_TBILL_MINT_OPERATOR_ROLE`
- `M_TBILL_BURN_OPERATOR_ROLE`
- `M_TBILL_PAUSE_OPERATOR_ROLE`

However, the core token contract `zHYPER.sol` enforces access control using entirely different role identifiers:

- `Z_HYPER_MINT_OPERATOR_ROLE`
- `Z_HYPER_BURN_OPERATOR_ROLE`
- `Z_HYPER_PAUSE_OPERATOR_ROLE`

Because the `keccak256` hashes of these role strings are different, the Admin is granted permissions for a non existent "M_TBILL" token but holds **zero operational permissions** for the actual `zHYPER` token.

Locations:

- `ZothAccessControl.sol`:84-87

Remediation:

Update `ZothAccessControl.sol` to import the correct role definitions (or define them locally if needed) and grant the `Z_HYPER` specific roles in `_setupRoles`.

Coded Fix:

```
_grantRole(Z_HYPER_MINT_OPERATOR_ROLE, admin);

_grantRole(Z_HYPER_BURN_OPERATOR_ROLE, admin);

_grantRole(Z_HYPER_PAUSE_OPERATOR_ROLE, admin);
```

Impact:

The impact of this misconfiguration is immediate and critical for protocol operations:

- The Admin cannot call `pause()` on the `zHYPER` token. The contract checks for `Z_HYPER_PAUSE_OPERATOR_ROLE`, but the Admin only holds `M_TBILL_PAUSE_OPERATOR_ROLE`. If an exploit is discovered, the protocol **cannot be paused**, allowing attackers to drain funds indefinitely while the Admin is locked out.
- The Admin (and by extension, any vault improperly relying on this setup) cannot mint tokens. While the `DepositVault` might be granted roles separately, any direct administrative minting (e.g., for liquidity provision or correction) will revert.
- The system believes it has initialized roles correctly, but the actual security state is effectively "Admin has no power." This creates a false sense of security where deployment scripts pass, but the system is operationally bricked.

Code Snippet:

```
_grantRole(M_TBILL_MINT_OPERATOR_ROLE, admin);

_grantRole(M_TBILL_BURN_OPERATOR_ROLE, admin);

_grantRole(M_TBILL_PAUSE_OPERATOR_ROLE, admin);
```

Reference:

NA

POC's Description:

NA

Proof of Vulnerability:

N.A.

4.3 Fiat Redemption Approval Reverts Due to Allowance Check on Zero Address

Severity	Status	Type of Analysis
● Medium	Acknowledged	Dynamic

Contract Name:

RedemptionVault

Description:

In the `RedemptionVault`, the `_approveRequest` function is responsible for finalizing redemptions. When calculating the payout, the function **unconditionally** calls `_requireAndUpdateAllowance(request.tokenOut, ...)` to ensure the Vault has enough liquidity to pay the user.

For **Fiat Redemptions**, the `tokenOut` is set to `MANUAL_FULLFILMENT_TOKEN` (which is `address(0)`).

1. The `tokensConfig[address(0)].allowance` is 0 by default (unless manually set by an admin).
2. The `_requireAndUpdateAllowance` function checks: `require(prevAllowance >= amount, "MV: exceed allowance")`.
3. Since `0 >= amount` is false (for any positive redemption amount), the transaction reverts.

This creates a logic conflict, fiat redemptions imply an off chain transfer, so they **do not consume on chain liquidity**. Therefore, they should not be subject to the Vault's token allowance logic.

Locations:

- [RedemptionVault.sol:346-356](#)

Remediation:

Modify the logic to **skip the allowance check** for Fiat redemptions. If an operational limit is desired for Fiat, implement a distinct `dailyFiatLimit` or similar variable that is semantically appropriate.

Client's Comments:

Risk Accepted by the client. Client opted for an operational fix rather than a code change. The Admin will manually set an infinite allowance for the zero address post-deployment.

Added comprehensive test coverage documenting the behavior.

Admin will call `changeTokenAllowance(address(0), maxUint256)` post-deployment

Impact:

- Fiat redemptions fail immediately upon deployment unless a specific, undocumented configuration step is taken.
- Error messages regarding Allowances (Liquidity) will be triggered for Fiat (Off chain) operations, confusing operators

about the Vault's solvency status.

Code Snippet:

```
function _approveRequest(

uint256 requestId

uint256 newZTokenRate

bool isSafe

) internal {

Request memory request = redeemRequests[requestId];

_validateRequest(request.sender, request.status);

if (isSafe) {

    _requireVariationTolerance(request.zTokenRate, newZTokenRate);

}

zToken.burn(address this, request.amountZToken);

bool isFiat = request.tokenOut == MANUAL_FULLFILMENT_TOKEN;

uint256 tokenDecimals = isFiat ? 18 : _tokenDecimals(request.tokenOut);

uint256 amountTokenOutWithoutFee = _truncate(

(request.amountZToken * newZTokenRate) / request.tokenOutRate,

tokenDecimals

);

_requireAndUpdateAllowance(request.tokenOut, amountTokenOutWithoutFee);

if (!isFiat) {

    _tokenTransferFromTo
```

```
request.tokenOut,  
  
requestRedeemer,  
  
request.sender,  
  
amountTokenOutWithoutFee,  
  
tokenDecimals  
  
);  
  
}  
  
request.status = RequestStatus Processed  
  
request.zTokenRate = newZTokenRate;  
  
redeemRequests[requestId] = request;  
  
}
```

Reference:

NA

POC's Description:

NA

Proof of Vulnerability:

N.A.

4.4 Non Standard Decimal Scaling Leads to Zero Value Deposits

Severity	Status	Type of Analysis
● Medium	Fixed	Dynamic

Contract Name:

DepositVault

Description:

The `DepositVault` and `RedemptionVault` contracts enforce a non standard input format. They expect all token amounts passed to functions like `depositInstant` or `depositRequest` to be **pre scaled to 18 decimals**, regardless of the underlying token's native decimals (e.g., USDC has 6 decimals, WBTC has 8).

The contracts handle the conversion internally using a down scaling logic during transfer:

1. User inputs `amount` (Expected: 18 decimals).
2. Contract calls `_tokenTransferFromUser`.
3. Logic performs `transferAmount = amount.convertFromBase18(tokenDecimals)`.
4. It transfers `transferAmount` from the user.

If a user or frontend/UI requests follows standard DeFi patterns and submits the **native token amount** (e.g., `1,000,000` for 1 USDC), the contract interprets this as `0.000000000001` (18 decimals).

- When the contract tries to convert this tiny amount back to USDC (6 decimals), the integer division results in **Zero**.
 - Math: $1,000,000 / 10^{(18-6)} = 1,000,000 / 1,000,000,000,000 = 0$
- The `safeTransferFrom` of 0 tokens usually succeeds. The user pays gas fees, but no deposit is recorded, and no tokens are moved.

Locations:

`DepositVault.sol:290`

Remediation:

Change function inputs to accept **native decimal amounts** and convert internally using `convertToBase18()` before calculations. This aligns with standard DeFi practices and prevents the truncation error.

Impact:

- Users interacting via direct calls to contract or custom scripts will likely input native amounts (standard practice). Their

transactions will execute successfully but result in 0 value credited.

- The frontend is forced to handle decimal scaling logic, which is typically the responsibility of the smart contract to normalize.
- Even if used correctly, the down scaling logic causes precision loss for tokens with >18 decimals (rare, but possible) or complex fractional amounts.

Code Snippet:

```
function depositRequest(  
  
    address tokenIn,  
  
    uint256 amountToken,  
  
    bytes32 referrerId,  
  
    address recipient  
  
)  
  
    external  
  
    whenFnNotPaused(_DEPOSIT_REQUEST_WITH_CUSTOM_RECIPIENT_SELECTOR)  
  
    returns (uint256 /*requestId*/)  
  
    {  
  
        _validateUserAccess(msg.sender);  
  
        if (recipient != msg.sender) {  
  
            _validateUserAccess(recipient);  
  
        }  
  
        (  
  
            uint256 requestId,  
  
            CalcAndValidateDepositResult memory calcResult  
  
        ) = _depositRequest(tokenIn, amountToken, recipient);  
  
    }
```

Reference:

NA

POC's Description:

NA

Proof of Vulnerability:

N.A.

4.5 PriceOracle Interface Mismatch (feedAdminRole() Missing)

Severity	Status	Type of Analysis
● Medium	Fixed	Dynamic

Contract Name:

IDataFeed

Description:

The protocol explicitly defines an `IDataFeed` interface that serves as the standard abstraction for all price feeds consumed by the Vaults. This interface mandates the implementation of two functions:

1. `getDataInBase18()`: Retrieves the price.
2. `feedAdminRole()`: Retrieves the access control role responsible for managing the feed.

However, the `PriceOracle` contract, which is intended to be the primary data feed for the system **only implements `getDataInBase18()`**. It completely fails to implement `feedAdminRole()`.

The `PriceOracle` inherits from `WithFunctionsAccessControl`, which manages roles internally, but it does not expose the `feedAdminRole` getter required by the `IDataFeed` interface. This creates a Zombie Implementation where the contract technically functions for fetching prices but crashes (reverts) immediately if any system component attempts to query its administrative configuration via the standard interface.

Locations:

- [IDataFeed.sol:14-26](#)
- [PriceOracle.sol](#)

Remediation:

Implement the missing `feedAdminRole` function in `PriceOracle.sol` to ensure full compliance with the `IDataFeed` interface.

example code fix:

In `zVault-dev/contracts/PriceOracle.sol`:

```
contract PriceOracle is WithFunctionsAccessControl {
    // ... existing storage and logic ...

    /**
     * @notice Constructor
     */
    constructor(...) {
        // ...
    }

    // ... existing functions ...

    /**
     * @notice FIX: Implement the missing interface function
     * Returns the role responsible for managing prices in this contract.
     */
    function feedAdminRole() external view returns (bytes32) {
        // Return the constant role defined in the inherited Access Control
        return accessControl.PRICE_ADMIN_ROLE();
    }
}
```

Impact:

- Any external tooling, admin dashboard, or bot designed to interact with the protocol's data feeds via the `IDataFeed` standard will fail. For example, a dashboard trying to display "Who is the admin for this feed?" will trigger a transaction revert when querying the `PriceOracle`.
- If a future upgrade to the Vault logic adds a security check that verifies the feed's admin (e.g., `require(IDataFeed(feed).feedAdminRole() == EXPECTED_ROLE)`), the entire Vault will brick itself because the `PriceOracle` cannot answer that query.
- The protocol violates its own internal standards. `IDataFeed` promises a specific behavior that the core implementation does not deliver.

Code Snippet:

```
interface IDataFeed {

    /**
     * @notice fetches answer from aggregator
     *
     * and converts it to the base18 precision
     *
     * @return answer fetched aggregator answer
     */

    function getDataInBase18() external view returns (uint256 answer);

    /**
     * @dev describes a role, owner of which can manage this feed
     *
     * @return role descriptor
     */

    function feedAdminRole() external view returns (bytes32);

}
```

Reference:

NA

POC's Description:

NA

Proof of Vulnerability:

N.A.

4.6 Sanctioned Users Can Bypass Bans via Token Transfers

Severity	Status	Type of Analysis
● Medium	Fixed	Dynamic

Contract Name:

zHYPER

Description:

The protocol uses a Sanctions Oracle to block banned users from using the system. This check works correctly at the Entry and "Exit" points, the [DepositVault](#) and [RedemptionVault](#) prevent sanctioned users from minting or redeeming.

However, the [zHYPER](#) token contract itself **does not check the Sanctions List** when users send tokens to each other. It only checks a manual Blacklist" which the admin has to update manually.

Since the token itself doesn't check sanctions, a sanctioned user can easily get around the block:

1. A user holds [zHYPER](#).
2. They get sanctioned (banned).
3. The Vault blocks them from redeeming for stablecoins.
4. They simply **transfer** their [zHYPER](#) to a different wallet that is not sanctioned.
5. That new wallet redeems the tokens for funds.

Locations:

- [zHYPER.sol](#)

Remediation:

Update the [zHYPER](#) token to use the [WithSanctionsList](#) module. Add the sanctions check to the transfer logic so that banned users are frozen completely.

example fix :

```
// 1. Import the Sanctions module
import "../abstract/WithSanctionsList.sol";

// 2. Add it to the contract inheritance
contract zHYPER is ERC20PausableUpgradeable, Blacklistable, WithSanctionsList {
```

```
// 3. Update Initialize to accept the Sanctions Oracle address
function initialize(
    address _accessControl,
    address _sanctionsList // New parameter
) external initializer {
    __Blacklistable_init(_accessControl);
    __WithSanctionsList_init(_sanctionsList); // Initialize the module
    __ERC20_init("Zoth Hyperithm", "zHYPER");
}

// 4. Update the Transfer Hook to check sanctions
function _beforeTokenTransfer(
    address from,
    address to,
    uint256 amount
) internal virtual override(ERC20PausableUpgradeable) {
    // Check if Sender or Receiver is sanctioned
    if (from != address(0)) {
        require(!isSanctioned(from), "zHYPER: sender is sanctioned");
        _onlyNotBlacklisted(from);
    }
    if (to != address(0)) {
        require(!isSanctioned(to), "zHYPER: recipient is sanctioned");
        _onlyNotBlacklisted(to);
    }

    ERC20PausableUpgradeable._beforeTokenTransfer(from, to, amount);
}
}
```

Impact:

- The system allows banned users to move their assets and cash out using a middleman, which likely violates regulatory rules.
- The Sanctions List is supposed to work automatically in real time. By ignoring it in the token contract, you lose this automatic protection and have to rely on the Admin manually blacklisting people one by one.

Code Snippet:

```
function _beforeTokenTransfer(  
  
    address from,  
  
    address to,  
  
    uint256 amount  
  
) internal virtual override ERC20PausableUpgradeable {  
  
    if (to != address 0) {  
  
        _onlyNotBlacklisted(from);  
  
        _onlyNotBlacklisted(to);  
  
    }  
  
    ERC20PausableUpgradeable _beforeTokenTransfer(from, to, amount);  
  
}
```

Reference:

NA

POC's Description:

NA

Proof of Vulnerability:

N.A.

4.7 Vaults Use Stale Prices if Oracle Stops Updating

Severity	Status	Type of Analysis
● Medium	Fixed	Dynamic

Contract Name:

PriceOracle

Description:

The protocol relies on a Push-based Oracle where an Admin manually updates the asset price using `setPrice`. The `PriceOracle` contract records the `lastUpdateTimestamp` whenever the price changes. The `PriceOracle` contract actually includes a safety function called `isPriceStale` designed specifically to check if the price data is too old.

However, the Vault contracts (`DepositVault` and `RedemptionVault`) **never call this function**. When they need to calculate an exchange rate, they call `_getTokenRate`, which fetches the price via `getDataInBase18()`. This function returns the price **regardless of how long ago it was set**.

If the Admin goes offline, the server crashes, or the private key is lost, the Oracle stops receiving updates. The Vaults will continue to value the assets at the *last known price*. If the real world market price moves significantly during this downtime, arbitrageurs can exploit the frozen price to drain value from the protocol.

Locations:

- [PriceOracle.sol:136-211](#)

Remediation:

Update the Vault logic to call `isPriceStale` (or check the timestamp directly) before using the price.

Coded Fix:

Step 1: Update the Interface In [zVault-dev/contracts/interfaces/IDataFeed.sol](#):

```
interface IDataFeed {
    function getDataInBase18() external view returns (uint256 answer);
    function feedAdminRole() external view returns (bytes32);

    // FIX: Add function to get the timestamp so Vaults can check it
    function getLastUpdateTimestamp() external view returns (uint256);
}
```

Step 2: Update the Oracle In [zVault-dev/contracts/PriceOracle.sol](#):

```
contract PriceOracle is WithFunctionsAccessControl {
    // ... existing code ...

    // FIX: Implement the interface function
    function getLastUpdateTimestamp() external view returns (uint256) {
        return lastUpdateTimestamp;
    }
}
```

Step 3: Enforce in Vaults In [zVault-dev/contracts/abstract/ManageableVault.sol](#):

```
// Add a config variable
uint256 public maxOracleStaleness; // e.g., 24 hours

function _getTokenRate(
    address dataFeed,
    bool stable
) internal view virtual returns (uint256) {
    if (stable) return STABLECOIN_RATE;

    // FIX: Check for staleness
    uint256 lastUpdate = IDataFeed(dataFeed).getLastUpdateTimestamp();

    require(lastUpdate > 0, "MV: Price never set");
    require(block.timestamp - lastUpdate <= maxOracleStaleness, "MV: Oracle price is stale");

    return IDataFeed(dataFeed).getDataInBase18();
}
```

Impact:

- If the real world price moves while the Oracle is frozen, users can trade against the old price to drain funds from the Vaults risk free.
- The protocol will value its assets incorrectly during downtime, potentially leading to insolvency if users redeem at inflated values.

Code Snippet:

```
function setPrice(
    uint256 _price

) external onlyRole(accessControl.PRICE_ADMIN_ROLE()) {

    if (_price == 0) revert InvalidPrice();

    // Convert input price to base18
```



```

// Convert input price to base18

uint256 newPriceBase18 = _convertToBase18(_price, priceDecimals);

// Check tolerance if there's a previous price and tolerance is set

if (currentPrice > 0 && tolerancePercent > 0) {

// Calculate percentage deviation in basis points

uint256 deviation;

if (newPriceBase18 > currentPrice) {

deviation =

((newPriceBase18 - currentPrice) * 10000) /

currentPrice;

} else {

deviation =

((currentPrice - newPriceBase18) * 10000) /

currentPrice;

}

// Revert if deviation exceeds tolerance

if (deviation > tolerancePercent) {

revert ToleranceExceeded(

currentPrice,

newPriceBase18,

deviation

);

}

}

```

```
uint256 oldPrice = currentPrice;  
  
currentPrice = newPriceBase18;  
  
lastUpdateTimestamp = block.timestamp;
```

Reference:

NA

POC's Description:

NA

Proof of Vulnerability:

N.A.

4.8 Redemption Request Can Record Zero Rates, Breaking Safe Approval

Severity	Status	Type of Analysis
● Low	Acknowledged	Dynamic

Contract Name:

RedemptionVault

Description:

In the `RedemptionVault` contract, the `redeemRequest` function retrieves the current exchange rates (`zTokenRate` and `tokenOutRate`) and stores them in the `Request` struct for future reference. However, the function **lacks validation checks** to ensure these rates are non-zero.

If the underlying Oracle feeds return 0 (e.g., due to downtime, uninitialized state, or misconfiguration), the request is successfully created with 0 rates. This creates a "poisoned" request that cannot be processed:

- Safe Approval Failure:** `safeApproveRequest` calculates price deviation using `_requireVariationTolerance`. This function calculates percentage difference by dividing by the `prevPrice` (the stored `zTokenRate`). If `zTokenRate` is 0, the transaction reverts due to **Division by Zero**.
- Calculation Failure:** Both safe and unsafe approvals divide by `request.tokenOutRate` to calculate the stablecoin payout. If `tokenOutRate` is 0, the approval reverts.

This creates a scenario where valid user requests become "stuck" and cannot be fulfilled, forcing the Admin to cancel (`rejectRequest`) them.

Locations:

- `RedemptionVault.sol:331-352`

Remediation:

Add explicit require checks inside `_redeemRequest` to prevent the creation of requests with invalid rates.

example code fix:

In `zVault-dev/contracts/RedemptionVault.sol`:

```
function _redeemRequest(
    address tokenOut,
    uint256 amountZTokenIn
) internal returns (uint256) {
    // ... (existing logic) ...

    address user = msg.sender;
    // ... fee calculations ...

    // 1. Fetch Rates
    uint256 zTokenRate = zTokenDataFeed.getDataInBase18();
    uint256 tokenOutRate = 1e18; // Default for Fiat

    bool isFiat = tokenOut == MANUAL_FULLFILMENT_TOKEN;
    if (!isFiat) {
        TokenConfig storage config = tokensConfig[tokenOut];
        tokenOutRate = _getTokenRate(config.dataFeed, config.stable);
    }

    // FIX: Validate Rates > 0
    require(zTokenRate > 0, "RV: zToken rate zero");
    require(tokenOutRate > 0, "RV: tokenOut rate zero");

    // ... (transfer logic and struct creation) ...
}
```

Client's Comments:

Risk Accepted by the client. Client chose to leave this as-is, relying on the Oracle's upstream validation to prevent zero values. The oracle contract already validates against zero values.

This is as intended by the contract design. The safe oracle check is only for instantDeposit and instantRedemption. Manual approval requires admin discretion.

Impact:

- Requests created during an Oracle outage cannot be approved, even after the Oracle comes back online.
- Admins must identify these stuck requests and manually reject them.
- Users experience indefinite delays and must restart the redemption process if their request is stuck.

Code Snippet:

```
/**
 * @notice Creating request depends on tokenOut
 * @param tokenOut tokenOut address
 * @param amountZTokenIn amount of zToken (decimals 18)
 *
 */
```

```
* @return requestId request id
```

```
*/
```

```
function _redeemRequest(
```

```
address tokenOut,
```

```
uint256 amountZTokenIn
```

```
) internal returns (uint256) {
```

```
address user = msg.sender;
```

```
bool isFiat = tokenOut == MANUAL_FULLFILMENT_TOKEN;
```

```
(
```

```
uint256 feeAmount,
```

```
uint256 amountZTokenWithoutFee
```

```
) = _calcAndValidateRedeem(
```

```
user,
```

```
tokenOut,
```

```
amountZTokenIn,
```

```
false,
```

```
isFiat
```

```
);
```

```
address tokenOutCopy = tokenOut;

// assigning the default value which is gonna be used

// only for fiat redemptions

uint256 tokenOutRate = 1e18;

if (!isFiat) {

    TokenConfig storage config = tokensConfig[tokenOutCopy];

    tokenOutRate = _getTokenRate(config.dataFeed, config.stable);

}

uint256 amountZTokenInCopy = amountZTokenIn;

uint256 zTokenRate = zTokenDataFeed.getDataInBase18();

_tokenTransferFromUser(

    address(zToken),

    address(this),

    amountZTokenWithoutFee,

    18 // zToken always have 18 decimals

);
```

```
if (feeAmount > 0)

_tokenTransferFromUser address(zToken), feeReceiver, feeAmount, 18);


uint256 requestId = currentRequestId.current();

currentRequestId.increment();


redeemRequests[requestId] = Request({
sender: user,

tokenOut: tokenOutCopy,

status: RequestStatus Pending

amountZToken: amountZTokenWithoutFee,

zTokenRate: zTokenRate,

tokenOutRate: tokenOutRate

});
```

Reference:

NA

POC's Description:

NA

Proof of Vulnerability:

N.A.

4.9 Redundant and Confusing mHYPER.sol Contract File

Severity	Status	Type of Analysis
● Low	Fixed	Dynamic

Contract Name:

mHYPER

Description:

The codebase contains a file located at `contracts/mHYPER/mHYPER.sol` that defines a contract named `zHYPER`. This file appears to be a leftover artifact, likely from a fork of the Protocol.

The content of this file is identical to the active token contract located at `contracts/zHYPER/zHYPER.sol`. Both files define a contract named `zHYPER` with the same logic and storage layout. The existence of two files defining the same contract name creates significant ambiguity.

Locations:

NA

Remediation:

Remove the `contracts/mHYPER` file and its contents entirely. Ensure all imports in the project point to `contracts/zHYPER/zHYPER.sol`.

Impact:

may generate warnings or unpredictable behavior when multiple files define artifacts with the exact same name (`zHYPER`).

Code Snippet:

NA

Reference:

NA

POC's Description:

NA

Proof of Vulnerability:

N.A.

4.10 Setting Price Tolerance to Zero Disables Safety Check Instead of Enforcing Exact Price

Severity	Status	Type of Analysis
● Low	Acknowledged	Dynamic

Contract Name:

PriceOracle

Description:

In `PriceOracle`, the protocol implements a deviation check to prevent massive price fluctuations during updates. The intention is to ensure the new price is within `tolerancePercent` of the old price.

However, the logic includes a conditional check that effectively disables the safety mechanism if the tolerance is set to `0`.

- Standard expectation: "Tolerance = 0" implies Zero Tolerance (i.e., the new price must match the old price exactly, or any deviation is rejected).
- Actual implementation: If `tolerancePercent` is `0`, the code **skips the check entirely**, allowing *infinite* deviation.

If an admin attempting to enforce strict security (by setting tolerance to 0), it will inadvertently remove all security, allowing any price to be set (e.g., via a compromised key or bug).

Locations:

PriceOracle: 104

Remediation:

Change the logic to allow `tolerancePercent == 0` to execute the check (enforcing strict equality). If the ability to *disable* the check is required, use a specific constant (e.g., `type(uint256).max`) or a separate boolean flag.

Client's Comments:

Risk Accepted by the Client. They have confirmed this is intentional design (0 means disabled, not exact match) and will rely on documentation.

Impact:

- An intentional attempt to lock down price updates results in the removal of all safeguards.
- The behavior of "0" is inconsistent with standard financial definitions of "Zero Tolerance."

Code Snippet:

```
*/  
  
function setPrice(  
  
    uint256 _price  
  
    ) external onlyRole(accessControl.PRICE_ADMIN_ROLE()) {  
  
    if (_price == 0) revert InvalidPrice();  
  
  
    // Convert input price to base18  
  
    uint256 newPriceBase18 = _convertToBase18(_price, priceDecimals);  
  
  
  
    // Check tolerance if there's a previous price and tolerance is set  
  
    if (currentPrice > 0 && tolerancePercent > 0) {  
  
        // Calculate percentage deviation in basis points  
  
        uint256 deviation;  
  
        if (newPriceBase18 > currentPrice) {  
  
            deviation =  
  
            ((newPriceBase18 - currentPrice) * 10000) /  
  
            currentPrice;  
  
        } else {  
  
            deviation =  
  
            ((currentPrice - newPriceBase18) * 10000) /  
  
            currentPrice;  
  

```

```
)  
  
// Revert if deviation exceeds tolerance  
  
if (deviation > tolerancePercent) {  
  
    revert ToleranceExceeded(  
  
        currentPrice,  
  
        newPriceBase18,  
  
        deviation  
  
    );  
}
```

Reference:

NA

POC's Description:

NA

Proof of Vulnerability:

N.A.

4.11 Admin approveRequest Bypasses Price Oracle Safety

Severity	Status	Type of Analysis
● Informational	Acknowledged	Dynamic

Contract Name:

DepositVault

Description:

The protocol asserts that it uses a `PriceOracle` with strict Variation Tolerance checks to prevent price manipulation. This is implemented in the `_requireVariationTolerance` function, which ensures the execution price doesn't deviate significantly from the request price.

However, the public-facing function `approveRequest` explicitly disables this safety check.

- The contract exposes two approval functions: `safeApproveRequest` and `approveRequest`.
- `safeApproveRequest` calls the internal logic with `isSafe = true`.
- `approveRequest` calls the internal logic with `isSafe = false`.

When `isSafe` is false, the protocol **completely skips the tolerance check**. This means the Admin can manually approve a deposit or redemption at *any* exchange rate they choose, disregarding the Oracle's current data.

Locations:

[DepositVault.sol](#): 547

Remediation:

Remove the unsafe `approveRequest` function entirely. The protocol should enforce the tolerance check by default. If an emergency override is truly needed for stuck requests, it should be gated behind a specific `EMERGENCY_ROLE` or a timelock, not available to the standard operational admin.

Client's Comments:

Risk Accepted by the client. They have confirmed this bypass is intentional to allow admins flexibility during edge cases or emergencies.

This is as intended by the contract design. The safe oracle check is only for `instantDeposit` and `instantRedemption`. Manual approval requires admin discretion.

Impact:

- If the `DEPOSIT_VAULT_ADMIN` private key is compromised, the attacker does not need to manipulate the Oracle. They can simply call `approveRequest` with a malicious rate (e.g., 1 Wei = 1 Million zHYPER) to mint infinite tokens instantly.
- The existence of this function renders the `PriceOracle`'s security guarantees null and void, as they can be legally bypassed by the Admin at any time.

Code Snippet:

```
function approveRequest(  
  
uint256 requestId  
  
uint256 newOutRate  
  
) external onlyVaultAdmin {  
  
    _approveRequest(requestId, newOutRate false true);  
  
    emit ApproveRequest(requestId, newOutRate);  
  
}
```

Reference:

NA

POC's Description:

NA

Proof of Vulnerability:

N.A.

5.0 Auditing Approach and Methodologies applied

The solidity smart contract was audited in a comprehensive approach to ensure the highest level of security and reliability. Careful attention was given to the following key areas to ensure the overall quality of code:

- **Code quality and structure:** We conducted a detailed review of the codebase to identify any potential issues related to code structure, readability, and maintainability. This included analyzing the overall architecture of the solidity smart contract and reviewing the code to ensure it follows best practices and coding standards.
- **Security vulnerabilities:** Our team used manual techniques to identify any potential security vulnerabilities that could be exploited by attackers. This involved a thorough analysis of the code to identify any potential weaknesses, such as buffer overflows, injection vulnerabilities, Signatures, and deprecated functions.

6.0 Limitations on Disclosure and Use of this Report

This report contains information concerning potential details of zVault and methods for exploiting them. Entersoft recommends that special precautions be taken to protect the confidentiality of both this document and the information contained herein. Security Assessment is an uncertain process, based on past experiences, currently available information, and known threats. All information security systems, which by their nature are dependent on human beings, are vulnerable to some degree. Therefore, while Entersoft considers the major security vulnerabilities of the analyzed systems to have been identified, there can be no assurance that any exercise of this nature will identify all possible vulnerabilities or propose exhaustive and operationally viable recommendations to mitigate those exposures. In addition, the analysis set forth herein is based on the technologies and known threats as of the date of this report. As technologies and risks change over time, the vulnerabilities associated with the operation of the Smart Contract described in this report, as well as the actions necessary to reduce the exposure to such vulnerabilities will also change. Entersoft makes no undertaking to supplement or update this report based on changed circumstances or facts of which Entersoft becomes aware after the date hereof, absent a specific written agreement to perform the supplemental or updated analysis. This report may recommend that Entersoft use certain software or hardware products manufactured or maintained by other vendors. Entersoft bases these recommendations upon its prior experience with the capabilities of those products. Nonetheless, Entersoft does not and cannot warrant that a particular product will work as advertised by the vendor, nor that it will operate in the manner intended. This report was prepared by Entersoft for the exclusive benefit of zVault and is proprietary information. The Non-Disclosure Agreement (NDA) in effect between Entersoft and zVault governs the disclosure of this report to all other parties including product vendors and suppliers.